



08. 상속과 다형성

Heesung Oh

<https://github.com/3dapi>



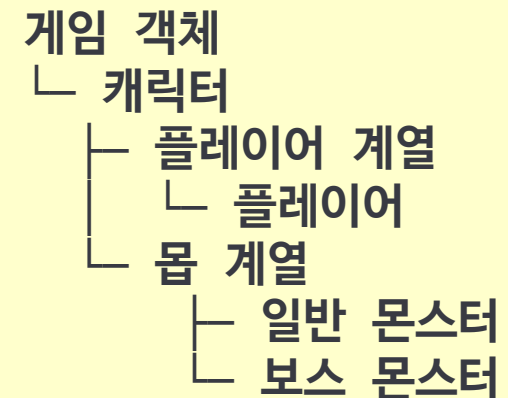
- 일반화와 상속
- 기반 클래스와 파생 클래스
- public / protected / private 상속
- 기반 클래스와 파생 클래스의 초기화
- 함수 재정의, 이름 숨김, override
- 가상 함수와 동적 바인딩
- 추상 클래스와 인터페이스 역할의 클래스
- 다형 객체의 수명과 가상 소멸자
- 업캐스팅, 다운캐스팅, RTTI
- 다중 상속과 가상 상속
- 상속과 구성의 선택





- 일반화: 여러 클래스의 공통 특성을 상위 개념으로 묶는 과정
- 상속: 하위 클래스가 상위 클래스의 특성을 이어받아 더 구체적인 클래스를 정의
- 공통 상태와 기능은 기반 클래스로 분리
- 구체적인 차이는 파생 클래스에서 표현

계층	역할
GameObject	게임 세계에 존재하는 객체의 공통 기반
Character	체력과 전투 상태를 가진 객체
PlayerBase	플레이어 계열의 공통 기반
MobBase	몹 계열의 공통 기반
Player, Monster, BossMonster	실제 생성 가능한 구체 클래스



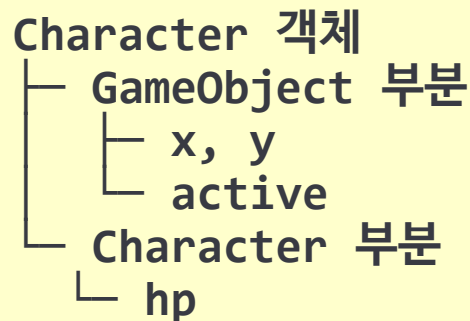


8.1.1 기반 클래스와 파생 클래스

- 기반 클래스: 공통 개념을 정의한 클래스
- 파생 클래스: 기반 클래스를 확장하거나 구체화한 클래스
- 파생 클래스 객체 안에는 기반 클래스 부분이 포함됨

```
class GameObject
{
public:
    void SetPosition(float x, float y);
    float GetX() const;
    float GetY() const;
protected:
    float x = 0.0f, y = 0.0f;
    bool active = true;
};
```

```
class Character : public
GameObject
{
public:
    void TakeDamage(int damage);
    int GetHp() const;
protected:
    int hp = 100;
};
```





8.1.1 다단계 상속 구조

```
class PlayerBase : public Character
{
public:
    void AddScore(int value);
    int GetScore() const;
protected:
    int score = 0;
};
```

```
class Player : public PlayerBase
{
};
```

```
class MobBase : public Character
{
public:
    void SetAttackPower(int attackPower);
    int GetAttackPower() const;
protected:
    int attackPower = 10;
};
```

```
class Monster : public MobBase
{
};
class BossMonster : public MobBase
{
};
```

Player 객체

- GameObject 부분
- Character 부분
- PlayerBase 부분
- Player 부분



- public 상속은 파생 클래스가 기반 클래스의 한 종류임을 표현
- 파생 객체는 기반 클래스 참조나 포인터가 필요한 위치에 전달 가능
- 문법상 변환뿐 아니라 기반 클래스의 의미와 규칙도 유지해야 함

표현	의미
Player is a Character	성립
Monster is a Character	성립
BossMonster is a GameObject	성립
Character is a Weapon	성립하지 않음

```
void PrintCharacterState(const Character& v)
{
    std::cout << v.GetX() << ", "
               << v.GetY() << ", hp="
               << v.GetHp() << "\n";
}
```

```
Player player;
Monster monster;
BossMonster boss;
PrintCharacterState(player);
PrintCharacterState(monster);
PrintCharacterState(boss);
```



- 공통 코드가 있다는 이유만으로 상속을 선택하지 않음
- 상속은 타입 관계와 인터페이스 계약을 함께 만듦
- 기반 클래스 대신 파생 클래스를 사용해도 의미가 유지되어야 함

기준	확인 내용
개념 관계	파생 클래스가 기반 클래스의 한 종류인가
인터페이스	기반 클래스의 공개 연산이 파생 클래스에도 의미가 있는가
계약	파생 클래스가 기반 클래스의 규칙을 유지하는가
다형성	구체 타입을 몰라도 공통 기능을 호출해야 하는가
변경 영향	기반 클래스 변경을 파생 클래스들이 수용할 수 있는가

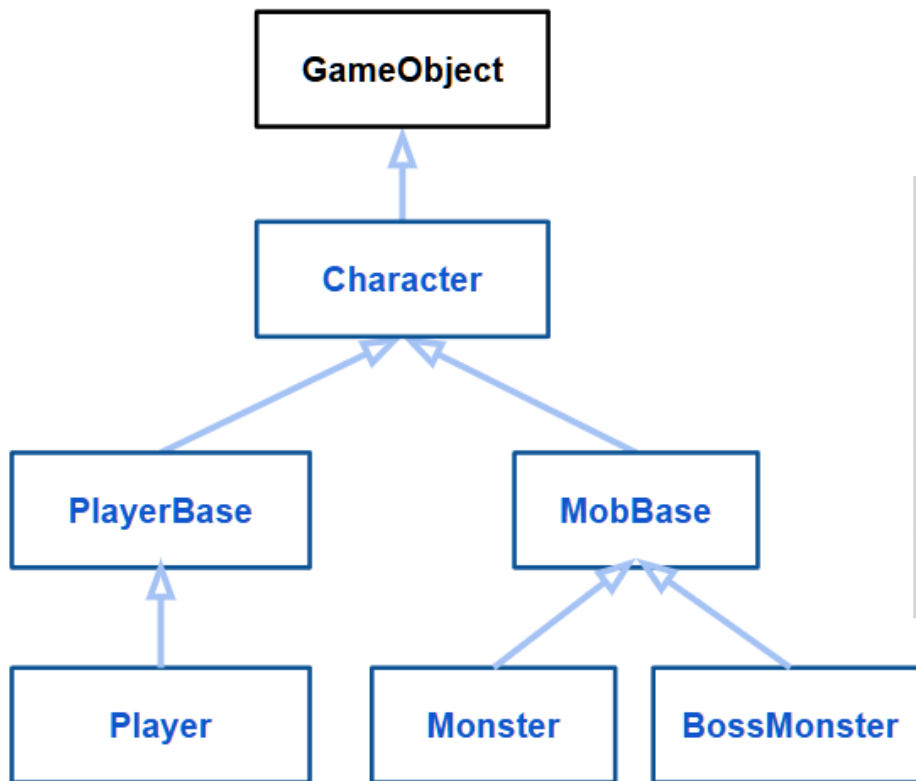
- **Character is a Weapon** → 잘못된 관계
- **Character has a Weapon** → 구성으로 표현





8.1.4 UML로 표현하는 일반화

- UML 다이어그램에서 일반화는 빈 삼각형 화살표로 표현
- 화살표 방향은 파생 클래스에서 기반 클래스를 향함
- UML은 설계 관계를 표현
- C++ 세부 구현은 별도 설명 필요



UML관계	C++ 표현
일반화	class Derived : public Base
합성	값 멤버, unique_ptr 등
연관	참조, 포인터 등
의존	매개변수, 지역 변수 등



- 파생 클래스 선언에서 기반 클래스 이름 앞에 상속 접근 지정자 작성
- 상속 접근 지정자는 기반 클래스의 public, protected 멤버가 파생 클래스에서 보이는 수준을 결정
- 기반 클래스의 private 멤버는 어떤 상속 방식에서도 파생 클래스가 직접 접근할 수 없음

```
class Derived : public Base { };
```

상속 방식	외부에 공개되는 관계	대표 목적
public	is-a 관계 공개	객체 계층과 다형성
protected	파생 계층 내부에서만 기반 관계 사용	제한된 구현 공유
private	기반 구현을 내부적으로 사용	구현 재사용





- public 상속은 공개적인 is-a 관계를 표현
- 기반 클래스의 public 멤버는 파생 클래스에서도 public 유지
- 기반 클래스의 protected 멤버는 파생 클래스에서도 protected 유지
- 외부 코드에서 업캐스팅 가능

```
class MobBase : public Character
{
};
```

```
class Monster : public MobBase
{
};
```

```
Monster monster;
monster.SetPosition(50.0f, 30.0f);
monster.TakeDamage(10);
```

```
MobBase& mob = monster;
Character& character = monster;
GameObject& object = monster;
```

기반 클래스 멤버	파생 클래스에서 접근 수준
public	public
protected	protected
private	직접 접근 불가



8.2.2 protected 상속과 private 상속

구분	protected 상속	private 상속
기반 public 멤버	protected가 됨	private가 됨
기반 protected 멤버	protected가 됨	private가 됨
외부 업캐스팅	일반적으로 불가	일반적으로 불가
대표 의미	파생 계층 내부 구현 공유	내부 구현 재사용

```
class InternalCharacter : protected Character
{
public:
    void ResetPosition()
    {
        SetPosition(0.0f, 0.0f);
    }
};
```

```
class PlayerController : private MovementService
{
public:
    void MovePlayer(float x, float y)
    {
        Move(x, y);
    }
};
```



8.2.4 protected 멤버

- protected 멤버는 클래스 외부에서는 접근 불가
- 파생 클래스의 멤버 함수에서는 접근 가능
- 데이터 멤버를 protected로 열면 파생 클래스가 내부 표현에 강하게 결합됨

```
class Character : public GameObject
{
protected:
    int hp = 100;
};

class MobBase : public Character
{
public:
    bool IsDefeated() const
    {
        return hp == 0;
    }
};
```

```
// 더 안전한 방향
class Character : public GameObject
{
public:
    int GetHp() const;

protected:
    void SetHp(int hp);

private:
    int hp = 100;
};
```



8.3 기반 클래스와 파생 클래스의 초기화

- 파생 클래스 객체를 생성하면 기반 클래스 부분도 함께 생성
- 기반 부분이 먼저 유효한 상태가 되어야 파생 부분 초기화 가능
- 생성은 계층의 위에서 아래로 진행
- 소멸은 계층의 아래에서 위로 진행

Player 객체 생성

GameObject



Character



PlayerBase



Player

BossMonster 객체 생성

GameObject



Character



MobBase



BossMonster



8.3.1 기반 클래스 생성자 호출

- 파생 클래스 생성자는 직접 기반 클래스의 생성자를 호출
- 직접 기반 클래스 생성자를 명시하지 않으면 기본 생성자를 호출
- 더 위의 기반 클래스는 직접 초기화할 수 없음

```
class GameObject
{
public:
    GameObject(int id, const std::string& name)
        : id(id), name(name)
    {
    }
private:
    int id;
    std::string name;
};
```

```
class Character : public GameObject
{
public:
    Character(int id, const std::string& name, int hp)
        : GameObject(id, name), hp(hp)
    {
    }
private:
    int hp;
};
```

- Character는 직접 기반 클래스인 GameObject를 초기화



- 각 클래스는 자신의 직접 기반 클래스만 선택

```
class PlayerBase : public Character
{
public:
    PlayerBase(int id,
               const std::string& name,
               int hp,
               int score)
        : Character(id, name, hp), score(score)
    {
    }

private:
    int score;
};
```

```
class Player : public PlayerBase
{
public:
    Player(int id, const std::string& name)
        // : GameObject(id, name) // 오류
        // : Character(id, name, 100) // 오류
        : PlayerBase(id, name, 100, 0)
    {
    }
};
```



8.3.3 초기화 순서

- 초기화 순서는 초기화 리스트가 아닌 클래스 선언 구조로 결정
- 초기화 리스트도 실제 초기화 순서와 맞추는 것이 좋음

```
class Player : public PlayerBase
{
public:
    Player(int id, const std::string& name, int level, int gold)
        : gold(gold),
          PlayerBase(id, name, 100, 0),
          level(level)
    {
    }

private:
    int level;
    int gold;
};
```

1. 가상 기반 클래스
2. 직접 기반 클래스
3. 데이터 멤버의 선언 순서
4. 생성자 본문

실제 순서

PlayerBase → level → gold → 생성자 본문



8.3.4 생성과 소멸 순서 예

```
class GameObject
{
public:
    GameObject() { std::cout << "GameObject 생성\n"; }
    virtual ~GameObject() { std::cout << "GameObject 소멸\n"; }
};

class Character : public GameObject
{
public:
    Character() { std::cout << "Character 생성\n"; }
    ~Character() override { std::cout << "Character 소멸\n"; }
};

class MobBase : public Character
{
public:
    MobBase() { std::cout << "MobBase 생성\n"; }
    ~MobBase() override { std::cout << "MobBase 소멸\n"; }
};

class BossMonster : public MobBase
{
public:
    BossMonster() { std::cout << "BossMonster 생성\n"; }
    ~BossMonster() override { std::cout << "BossMonster 소멸\n"; }
};
```

과정	방향
생성	기반 클래스 → 파생 클래스
소멸	파생 클래스 → 기반 클래스





8.4 함수 재정의와 이름 숨김

- 파생 클래스는 기반 클래스에서 물려받은 함수와 같은 이름의 함수를 선언할 수 있음
- 오버로딩, 오버라이딩, 이름 숨김을 구분해야 함

구분	의미
오버로딩	같은 범위에서 이름은 같고 매개변수 목록이 다른 함수 선언
오버라이딩	파생 클래스가 기반 클래스의 가상 함수를 같은 시그니처로 재정의
이름 숨김	파생 클래스의 같은 이름이 기반 클래스의 이름 검색을 가림

```
class Character
{
public:
    void Move(float x, float y);
    void Move(float distance);
};
```

```
class Player : public Character
{
public:
    void Move(int direction);
};
```

- Player::Move(int)가 Character::Move 오버로드 집합을 숨길 수 있음



- 재정의 함수에 **override** 를 붙여 시그니처 오류를 검사

구분	오버로딩	오버라이딩
관계	같은 범위의 함수들	기반 클래스와 파생 클래스
매개변수	서로 달라야 함	같아야 함
virtual	필요 없음	기반 함수에 필요
선택 시점	주로 컴파일 시점	기반 포인터·참조에서는 실행 시점

```
class GameObject
{
public:
    virtual void Update()
    {
        std::cout << "GameObject Update\n";
    }
};
```

```
class Monster : public GameObject
{
public:
    void Update() override
    {
        std::cout << "Monster AI Update\n";
    }
};
```



8.4.2 이름 숨김과 using 선언

```
class Character : public GameObject
{
public:
    void Move(float x, float y)
    {
        std::cout << "Character Move\n";
    }
};
```

```
class Player : public Character
{
public:
    void Move(int direction)
    {
        std::cout << "Player input direction\n";
    }
};
```

```
Player player;
player.Move(1);
// player.Move(10.0f, 20.0f); // 오류
```

// 기반 함수 다시 노출

```
class Player : public Character
{
public:
    using Character::Move;

    void Move(int direction);
};
```

- using Character::Move;는 기반 클래스의 오버로드 집합을 다시 보이게 함



- 파생 클래스가 기반 클래스 구현을 확장할 때 범위 지정 연산자 사용
- 범위를 명시한 호출은 해당 클래스의 구현을 직접 호출

```
class Character : public GameObject
{
public:
    void Update() override
    {
        GameObject::Update();
        std::cout << "Character status update\n";
    }
};
```

```
class Player : public Character
{
public:
    void Update() override
    {
        Character::Update();
        std::cout << "Player animation update\n";
    }
};
```

GameObject Update
Character status update
Player animation update

8.4.5 재정의의 시그니처와 override

- 가상 함수 재정의 ← 시그니처 일치
- 이름, 매개변수 타입, 순서, const, 참조 한정자 등 재정의의 조건에 영향
- override** 는 재정의 여부를 컴파일러가 검사하게 함

```
class MobBase : public Character
{
public:
    virtual void Think() const
    {
    }
};
```

```
class Monster : public MobBase
{
public:
    void Think() const override
    {
        std::cout << "Monster patrol\n";
    }
};
```

```
class BossMonster : public MobBase
{
public:
    // void Think() override; // 오류: const가 일치하지 않음
};
```



- 기반 클래스 포인터나 참조가 실제로 가리키는 객체에 따라 다른 함수 호출
- 가상 함수는 실행 시점의 실제 객체 타입을 사용
- 다형성의 핵심 기능

구분	의미
정적 타입	변수나 표현식 선언에서 결정되는 타입
동적 타입	실행 중 실제로 존재하는 객체의 타입
정적 바인딩	컴파일 시점에 호출 함수 결정
동적 바인딩	실행 시점의 동적 타입에 따라 호출 함수 결정

```
GameObject& object = player;
```

정적 타입: **GameObject&**

동적 타입: **Player**



● 비가상 함수

```
class GameObject
{
public:
    void PrintType() const
    {
        std::cout << "GameObject\n";
    }
};
```

```
class Player : public GameObject
{
public:
    void PrintType() const
    {
        std::cout << "Player\n";
    }
};
```

```
Player player;
GameObject& object = player;
object.PrintType(); // GameObject
```

● 가상 함수

- ◆ 가상 함수 호출은 실제 객체의 동적 타입을 반영

```
class GameObject
{
public:
    virtual void PrintType() const;
}
```

```
class Player : public GameObject
{
public:
    void PrintType() const override;
};
```




- 같은 형식으로 호출, 실행은 동적 타입에 따라 달라짐

```
class GameObject
{
public:
    virtual ~GameObject() = default;
    virtual void Update()
    {
        std::cout << "GameObject Update\n";
    }
    virtual void Render() const
    {
        std::cout << "GameObject Render\n";
    }
};
```

```
class Player : public GameObject
{
public:
    void Update() override
    {
        std::cout << "Player Update\n";
    }
    void Render() const override
    {
        std::cout << "Player Render\n";
    }
};
```

```
void Process(GameObject& object)
{
    object.Update();
    object.Render();
}
```



- **override** : 기반 가상 함수를 재정의한다는 의도 명시
- **final**
 - ◆ 더 이상 재정의하거나 상속하지 못하게 제한
 - ◆ final은 확장을 막는 강한 제약
 - ◆ 명확한 이유가 있을 때 사용

```
class Monster : public MobBase
{
public:
    void Update() override
    {
    }
};
```

```
class BossMonster : public MobBase
{
public:
    void EnterRageMode() final
    {
        std::cout << "Boss rage mode\n";
    }
};
```

```
class FinalBossMonster final : public BossMonster
{
};
```

// class HiddenBoss : public FinalBossMonster // 오류



- C++ 표준은 구현 방식으로 vtable을 요구하지 않음
- 대부분의 구현은 vtable과 vptr에 가까운 구조 사용
- 개념 이해용 모델로 활용

GameObject 가상 함수 테이블

- Update → GameObject::Update
- Render → GameObject::Render
- 소멸자 → GameObject::~~GameObject

Player 가상 함수 테이블

- Update → Player::Update
- Render → Player::Render
- 소멸자 → Player::~~Player

Player 객체

- vptr → Player 가상 함수 테이블
- GameObject 부분
- Character 부분
- PlayerBase 부분
- Player 부분



8.5.5 공변 반환형 (covariant return type)

- 기반 클래스의 포인터나 참조를 반환하는 가상 함수
- 공변 반환형
 - ◆ 파생 클래스 재정의 함수에서 더 구체적인 포인터나 참조를 반환

```
class GameObject
{
public:
    virtual ~GameObject() = default;
    virtual GameObject* CloneRaw() const
    {
        return new GameObject(*this);
    }
};
```

```
class Player : public PlayerBase
{
public:
    Player* CloneRaw() const override
    {
        return new Player(*this);
    }
};
```

스마트 포인터 템플릿 타입에는 공변 반환형 규칙이 적용되지 않음

```
// std::unique_ptr<Player>는  
// std::unique_ptr<GameObject>의 공변 반환형이 아님
```



8.5.6 생성자와 소멸자에서의 가상 함수 호출

- 생성자와 소멸자 안의 가상 호출은 현재 생성 또는 소멸 중인 클래스 범위를 기준으로 처리
- 아직 생성되지 않은 파생 부분을 사용하지 않도록 하기 위한 규칙
- 이미 소멸한 파생 부분을 사용하지 않도록 하기 위한 규칙

```
class GameObject
{
public:
    GameObject()
    {
        Initialize();
    }
    virtual ~GameObject()
    {
        Shutdown();
    }
    virtual void Initialize();
    virtual void Shutdown();
};

class Player : public PlayerBase
{
public:
    void Initialize() override;
    void Shutdown() override;
};
```

- Player 생성 중 GameObject 생성자에서는 GameObject::Initialize() 호출
- 파생 초기화는 객체 생성 완료 뒤 별도 단계로 처리



- 가상 함수 구현은 동적 타입에 따라 선택
- 기본 인수는 호출 표현식의 정적 타입에서 선택

```
class GameObject
{
public:
    virtual void Draw(int layer = 0) const;
};

class Player : public PlayerBase
{
public:
    void Draw(int layer = 10) const override;
};
```

```
Player player;
GameObject& object = player;
```

```
player.Draw();
object.Draw();
```

```
Player layer: 10
Player layer: 0
```

- 두 호출 모두 Player::Draw() 실행
- object.Draw()의 기본 인수는 GameObject 선언에서 가져옴
- 가상 함수마다 다른 기본 인수를 두지 않는 편이 좋음



- 공통 계약만 선언하고 실제 동작은 파생 클래스에 맡길 수 있음
- 순수 가상 함수는 = 0으로 선언
- 순수 가상 함수가 하나 이상 있으면 추상 클래스
- 추상 클래스는 직접 객체 생성 불가

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual void Update() = 0;
    virtual void Render() const = 0;
};
```

```
// GameObject object; // 오류
```

구분	의미
순수 가상 함수	파생 클래스가 구현해야 하는 계약
추상 클래스	직접 객체를 만들 수 없는 공통 기반
구체 클래스	모든 순수 가상 함수를 구현한 클래스



- 추상 클래스는 공통 상태와 일부 구현을 가질 수 있음
- 생성자, 데이터 멤버, 일반 멤버 함수 포함 가능
- 순수 가상 함수를 구현하지 않으면 계속 추상 클래스

```
class Character : public GameObject
{
public:
    explicit Character(int hp) : hp(hp){}
    void TakeDamage(int damage)
    {
        if (damage > 0)
        {
            if (hp -= damage; hp < 0)
            {
                hp = 0;
            }
        }
    }
    int GetHp() const
    {
        return hp;
    }
private:
    int hp;
};
```

- Update(), Render()를 구현하지 않았다면 Character도 추상 클래스



8.6.3 인터페이스 역할의 클래스

- C++에는 별도 interface 키워드가 없음
 - ◆ ※일부에서 순수 가상함수로 구성된 클래스를 interface 로 부르기도 함
- 순수 가상 함수와 가상 소멸자로 인터페이스 역할의 클래스 구성 가능
- 데이터 멤버 없이 작은 계약으로 유지하는 편이 좋음

```
class IDamageable
{
public:
    virtual ~IDamageable() = default;
    virtual void TakeDamage(int damage) = 0;
};
```

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};
```

```
class Character : public GameObject,
                 public IDamageable
{
public:
    void TakeDamage(int damage) override;
    void Save(std::ostream& output) const override;
```



8.6.4 구현 상속과 인터페이스 상속

구분	구현 상속	인터페이스 상속
목적	상태와 구현 재사용	공통 호출 계약 제공
데이터 멤버	가질 수 있음	보통 두지 않음
일반 함수	포함 가능	순수 가상 함수 중심
결합도	기반 구현에 강하게 결합	계약에 상대적으로 약하게 결합

```
class MobBase : public Character
{
protected:
    int attackPower = 10;
};
```

```
class BossMonster : public MobBase,
                    public ISavable
{
public:
    void Save(std::ostream& output) const override
    {
        output << "BossMonster\n";
    }
};
```

- 하나의 주 구현 계층 + 여러 작은 인터페이스 구조가 관리하기 쉬움



- 기반 클래스 **공개 인터페이스**는 **모든 파생 클래스에 의미가 있어야**
- 특정 파생 클래스에서만 필요한 기능을 최상위 기반에 넣지 않음
- 서로 독립적인 역할은 작은 인터페이스로 분리

```
class GameObject
{
public:
    // virtual void ProcessInput() = 0;
    // Monster에 부적절
};
```

```
class PlayerBase : public Character
{
public:
    virtual void ProcessInput() = 0;
};
```

```
class MobBase : public Character
{
public:
    virtual void Think() = 0;
};
```

기준	설명
의미	모든 파생 클래스에 의미 있는가
계약	사전 조건과 사후 조건을 유지할 수 있는가
안정성	자주 바뀌지 않는가
크기	필요한 연산만 포함하는가



- 공개 함수는 비가상으로 유지
- 내부에서 보호 가상 함수를 호출
- 공통 규칙을 기반 클래스가 보장
- 파생 클래스는 확장 지점만 구현

```
class GameObject
{
public:
    virtual ~GameObject() = default;
    void Update()
    {
        if (!active) {
            return;
        }
        OnUpdate();
    }
protected:
    virtual void OnUpdate() = 0;
private:
    bool active = true;
};
```

```
class Monster : public MobBase
{
protected:
    void OnUpdate() override
    {
        Think();
    }
};
```

- 파생 클래스가 공통 검사를 빠뜨릴 가능성을 줄임



- 다형 객체 → 파생 클래스 객체를 기반 클래스 포인터, 참조로 활용
- 다형 객체를 기반 클래스 포인터로 관리할 때 수명 설계가 중요
- 기반 클래스 소멸자는 가상이어야 함
- 객체 잘림 방지 필요
- 다형 복사는 Clone 패턴으로 처리 가능
- 스마트 포인터 컨테이너로 소유권을 표현

문제	대응
기반 포인터 삭제	가상 소멸자
기반 값 복사	참조·포인터 전달
컨테이너 저장	<code>std::unique_ptr<Base></code>
동적 타입 유지 복사	가상 Clone()
원시 포인터 소유	스마트 포인터 사용



- 기반 클래스 포인터를 통해 파생 객체를 삭제할 가능성이 있으면 기반 클래스 소멸자는 가상
- 파생 클래스 소멸자는 자동으로 가상 함수
- `override` 를 소멸자에도 사용할 수 있음

```
class GameObject
{
public:
    virtual ~GameObject() = default;
};
class Character : public GameObject
{
public:
    ~Character() override = default;
};
```

```
GameObject* object = new BossMonster;
delete object;
```

BossMonster 소멸
MobBase 소멸
Character 소멸
GameObject 소멸

- 기반 소멸자가 비가상이면 기반 포인터 삭제는 정의되지 않은 동작



8.7.3 객체 잘림 (Object slicing)

- 파생 객체를 기반 클래스 값으로 복사하면 기반 클래스 부분만 복사됨
- 파생 클래스 부분은 사라짐
- 이를 객체 잘림이라고 함

```
class Character
{
public:
    virtual ~Character() = default;

    virtual void PrintType() const
    {
        std::cout << "Character\n";
    }
};

class Player : public Character
{
public:
    void PrintType() const override
    {
        std::cout << "Player\n";
    }
};
```

```
Player player;
Character copied = player;

copied.PrintType(); // Character
```

- 다형 객체는 값이 아니라 참조나 포인터로 전달



- 기반 클래스 포인터가 가리키는 구체 타입을 유지하며 복사하려면 가상 복제 함수 제공
- 반환 타입은 기반 스마트 포인터로 통일

```
class GameObject {  
public:  
    virtual ~GameObject() = default;  
    virtual std::unique_ptr<GameObject> Clone() const = 0;
```

```
class Player : public PlayerBase {  
public:  
    std::unique_ptr<GameObject> Clone() const override {  
        return std::make_unique<Player>(*this);  
    }  
}
```

```
class BossMonster : public MobBase {  
public:  
    std::unique_ptr<GameObject> Clone() const override {  
        return std::make_unique<BossMonster>(*this);  
    }  
}
```

```
std::unique_ptr<GameObject> copied = original->Clone();
```




- 고유 소유권을 가진 다형 객체는 `std::unique_ptr<Base>`로 관리
- 컨테이너에 기반 클래스 값을 직접 저장하지 않음
- 객체 잘림 없이 동적 타입 유지

```
std::vector<std::unique_ptr<GameObject>> objects;
```

```
objects.push_back(std::make_unique<Player>());  
objects.push_back(std::make_unique<Monster>());  
objects.push_back(std::make_unique<BossMonster>());
```

```
for (const auto& object : objects)  
{  
    object->Update();  
    object->Render();  
}
```

```
std::unique_ptr<Player> concrete = std::make_unique<Player>();  
std::unique_ptr<GameObject> object = std::move(concrete);
```



- 상속 계층의 포인터와 참조는 업캐스팅과 다운캐스팅을 지원
- RTTI는 실행 중 실제 타입을 확인하는 기능

변환	의미	안전성
업캐스팅	파생 → 기반	공개 상속에서는 일반적으로 안전
다운캐스팅	기반 → 파생	실제 타입 확인 필요
static_cast	실행 중 검사 없음	타입 보장 없으면 위험
dynamic_cast	실행 중 타입 검사	다형 기반 클래스 필요
typeid	타입 정보 확인	문자열 형식은 구현 의존





- 파생 클래스 포인터나 참조를 접근 가능한 기반 클래스 포인터나 참조로 변환
- 공개 상속에서는 일반적으로 암시적으로 수행 가능
- 업캐스팅 후에는 정적 타입이 제공하는 인터페이스만 사용 가능

```
Player player;
```

```
PlayerBase* playerBase = &player;
```

```
Character* character = &player;
```

```
GameObject* object = &player;
```

```
BossMonster boss;
```

```
MobBase& mob = boss;
```

```
Character& character = boss;
```

```
GameObject& object = boss;
```

```
GameObject& object = boss;
```

```
object.Update();
```

```
// object.EnterRageMode(); // 오류
```





- 기반 클래스 포인터나 참조를 더 구체적인 파생 클래스 타입으로 변환
- **static_cast**: 실행 중 타입을 검사하지 않음
- **dynamic_cast**:
 - ◆ 다형 기반 클래스의 RTTI를 사용하여 타입 검사 → 실패 시 **nullptr** 반환

```
void ActivateBossMode(GameObject* object) {  
    BossMonster* boss = dynamic_cast<BossMonster*>(object);  
    if (boss != nullptr) {  
        boss->EnterRageMode();  
    }  
}  
  
try  
{  
    BossMonster& boss = dynamic_cast<BossMonster&>(object);  
    boss.EnterRageMode();  
}  
catch (const std::bad_cast& error)  
{  
    std::cout << error.what() << '\n';  
}
```



- 다운캐스팅이 반복되면 기반 인터페이스가 필요한 행동을 충분히 표현하지 못할 수 있음
- 타입 검사 분기보다 가상 함수나 작은 인터페이스로 표현

```
void ExecuteAction(GameObject& object)
{
    if (auto* monster = dynamic_cast<Monster*>(&object))
    {
        monster->Attack();
    }
    else if (auto* boss = dynamic_cast<BossMonster*>(&object))
    {
        boss->Attack();
    }
}
```

```
class MobBase : public Character
{
public:
    virtual void Attack(Character& target) = 0;
};

void ExecuteAttack(MobBase& mob, Character& target)
{
    mob.Attack(target);
}
```



- 한 파생 클래스가 둘 이상의 기반 클래스 상속
- 여러 역할을 하나의 타입에 결합 가능
- 상태 중복, 이름 충돌, 다이아몬드 상속 문제 발생 가능

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};

class BossMonster : public MobBase,
                    public ISavable
{
public:
    void Save(std::ostream& output) const override
    {
        output << "BossMonster\n";
    }
};
```

형태	위험
주 구현 계층 + 인터페이스	비교적 단순
여러 상태 기반 클래스	이름 충돌, 상태 중복
다이아몬드 구조	기반 부분 중복, 초기화 복잡



- 둘 이상의 기반 클래스에 같은 이름의 멤버가 있으면 모호성 발생
- 범위를 명시하거나 파생 클래스에서 의미를 통합
- 이름 충돌이 잦으면 인터페이스 책임이 겹치는지 검토

```
class BossMonster : public MobBase,  
                    public IRenderable,  
                    public ISavable  
{  
};
```

```
BossMonster boss;  
// boss.Reset(); // 오류: 어느 기반 함수인지 모호함
```

```
// 명시적 호출  
boss.IRenderable::Reset();  
boss.ISavable::Reset();
```



- 두 기반 클래스가 같은 상위 기반 클래스를 상속
- 하나의 파생 클래스가 두 기반 클래스를 모두 상속
- 최상위 기반 부분이 두 개 생길 수 있음

```
class GameObject
{
public:
    int GetId() const;
};

class Character: public GameObject
{};

class NetworkObject: public GameObject
{};

class NetworkCharacter: public Character,
                        public NetworkObject
{};
```

```
NetworkCharacter object;
// object.GetId(); // 오류: 모호함
```

NetworkCharacter 객체
├── Character 부분
│ ├── GameObject 부분
│ └── NetworkObject 부분
└── NetworkObject 부분
 └── GameObject 부분



- 다이아몬드 구조에서 최상위 기반 클래스 부분을 하나만 공유
- **virtual** 을 추가, 기반 부분 중복을 해결
- 객체 배치, 초기화, 접근 비용은 더 복잡해질 수 있음

```
class Character: public virtual GameObject
{
};

class NetworkObject: public virtual GameObject
{
};

class NetworkCharacter: public Character, public NetworkObject
{
};
```

// 구성으로 대체 가능

```
class NetworkComponent
{
public:
    void Replicate();
};
```

```
class Character : public GameObject
{
private:
    NetworkComponent network;
};
```



8.9.5 가상 기반 클래스 초기화

- 가상 기반 클래스는 가장 구체적인 파생 클래스가 직접 초기화
- 중간 클래스의 가상 기반 초기화 지정은 최종 객체 생성에서는 사용되지 않을 수 있음

항목	규칙
초기화 책임	가장 구체적인 파생 클래스
생성 순서	가상 기반 → 일반 기반 → 멤버 → 생성자 본문
중복 기반 부분	하나만 존재

```
class NetworkCharacter : public Character,  
                        public NetworkObject  
{  
public:  
    NetworkCharacter()  
        : GameObject(100), Character(), NetworkObject()  
    {  
    }  
};
```

- NetworkCharacter가 가장 구체적인 파생 클래스이므로 가상 기반 클래스 GameObject를 직접 초기화
- Character, NetworkObject에서 지정한 GameObject 초기화보다 GameObject(100)이 적용됨



8.9.6 인터페이스 다중 상속

- 상태와 구현을 가진 여러 클래스를 상속하면 복잡도 증가
- 작은 인터페이스 여러 개를 구현하는 방식은 비교적 안전
- 호출 코드는 필요한 역할에만 의존

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};
```

```
class ITargetable
{
public:
    virtual ~ITargetable() = default;
    virtual bool CanBeTargeted() const = 0;
};
```

```
class BossMonster : public MobBase,
                    public ISavable,
                    public ITargetable
{
public:
    void Save(std::ostream& output) const override;
    bool CanBeTargeted() const override;
};
```



- 상속은 is-a 관계
- 구성은 has-a 관계
- 두 방식은 코드 재사용 수단이 아니라 객체 관계와 책임 설계의 선택

관계	C++ 표현	의미
is-a	public 상속	기반 타입으로 대체 가능
has-a	멤버 객체	객체가 기능이나 상태를 가짐
uses	참조, 포인터, 매개변수	일시적 사용 또는 비 소유 접근
owns	값, unique_ptr, 컨테이너	수명 책임

상속

Player is a Character.
Monster is a
Character.

구성

Player has an Inventory.
Player has a Weapon.
BossMonster has a
BossPhaseController.



- 구성 객체는 자신의 상태와 규칙을 관리
- 필요한 기능만 외부에 제공

```
class Inventory
{
public:
    void AddItem(int itemId)
    {
        items.push_back(itemId);
    }

private:
    std::vector<int> items;
};

class Weapon
{
public:
    void Attack()
    {
        std::cout << "Weapon attack\n";
    }
};
```

```
class Player : public PlayerBase
{
public:
    void AddItem(int itemId)
    {
        inventory.AddItem(itemId);
    }

    void Attack()
    {
        weapon.Attack();
    }

private:
    Inventory inventory;
    Weapon weapon;
};
```



- 객체가 자신이 가진 다른 객체에 작업을 맡기는 것
- 기능을 독립된 구성 요소로 분리 가능

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp);
    void TakeDamage(int damage);
    int GetHp() const;

private:
    int hp;
    int maxHp;
};
```

```
class Character : public GameObject
{
public:
    explicit Character(int hp)
        : health(hp)
    {
    }

    void TakeDamage(int damage)
    {
        health.TakeDamage(damage);
    }

    int GetHp() const
    {
        return health.GetHp();
    }

private:
    HealthComponent health;
};
```



- 파생 클래스는 기반 클래스의 인터페이스, 초기화 규칙, 수명에 의존
- protected** 데이터에 직접 접근하면 내부 표현에도 의존

```
class MobBase : public Character
{
protected:
    void RecoverHalfHp()
    {
        hp += 50; // Character의 저장 방식에 직접 의존
    }
};
```

```
// Character의 저장 방식에 직접 의존
class Character : public GameObject
{
public:
    int GetHp() const;
    void Recover(int amount);

private:
    HealthComponent health;
};

class MobBase : public Character
{
protected:
    void RecoverHalfHp()
    {
        Recover(50);
    }
};
```



8.10.4 상속보다 구성을 우선하는 이유

- 실행 중 구성 요소 교체 가능
- 기능 조합 때문에 파생 클래스 수가 늘어나는 문제 감소
- 기반 구현 변경의 영향 제한
- 각 구성 요소를 독립적으로 테스트 가능

```
class IMobBehavior
{
public:
    virtual ~IMobBehavior() = default;
    virtual void Think() = 0;
};

class Monster : public MobBase
{
public:
    explicit Monster(std::unique_ptr<IMobBehavior>
behavior)
        : behavior(std::move(behavior)){ }
    void Think() override
    {
        behavior->Think();
    }
private:
    std::unique_ptr<IMobBehavior> behavior;
};
```

상속으로 행동 분류

MobBase

- PatrolMonster
- ChaseMonster
- RangedMonster

PatrolRangedMonster

- ChaseRangedMonster



- 공개적인 타입 관계와 동적 대체가 필요하면 상속이 적합
- 구체 타입을 몰라도 공통 인터페이스로 처리해야 하는 구조

상속이 적합한 경우	구성이 적합한 경우
파생 객체가 기반 객체의 한 종류	객체가 다른 기능 객체를 소유
공통 인터페이스로 여러 구체 타입 처리	기능을 실행 중 교체
기반 클래스 계약이 안정적	여러 기능의 조합 필요
동적 바인딩 필요	타입 관계보다 구현 재사용이 목적
프레임워크 확장 지점	기반 변경과의 결합을 줄여야 함





- GameScene은 서로 다른 게임 객체를 고유 소유
- 컨테이너에는 `std::unique_ptr<GameObject>` 저장
- GameObject는 추상 기반 클래스
- 구체 클래스는 `Update()`, `Render()`, `Clone()` 구현
- 체력은 `HealthComponent`가 관리
- 프레임 시간은 `GameSystem`에서 조회





- 직접 객체를 만들 수 없는 추상 클래스
- 다형 소멸을 위한 가상 소멸자 제공



8.11.3 Character와 Player

```
class Character : public GameObject
{
public:
    Character(int id, std::string name, int maxHp)
        : GameObject(id, std::move(name)), health(maxHp)
    {
    }

    void TakeDamage(int damage)
    {
        health.TakeDamage(damage);
    }

    int GetHp() const
    {
        return health.GetHp();
    }

private:
    HealthComponent health;
};
```

```
class Player : public PlayerBase
{
public:
    void Update() override;
    void Render() const override;

    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<Player>(*this);
    }
};
```



8.11.4 MobBase, Monster, BossMonster

- 몹 계열의 공통 계약은 MobBase에 위치
- 구체 몬스터는 각자의 판단과 공격 구현

```
class MobBase : public Character
{
public:
    virtual void Think() = 0;
    virtual void Attack(Character& target) = 0;

protected:
    int GetAttackPower() const
    {
        return attackPower;
    }

private:
    int attackPower = 10;
};
```

```
class Monster : public MobBase
{
public:
    void Think() override;
    void Attack(Character& target) override;
    void Update() override;
    void Render() const override;
    std::unique_ptr<GameObject> Clone()
    const override;
};
```

```
class BossMonster : public MobBase
{
public:
    void Think() override;
    void Attack(Character& target) override;
    void EnterRageMode();
};
```



8.11.5 GameScene의 다형 객체 소유

```
class GameScene
{
public:
    void Add(std::unique_ptr<GameObject> object)
    {
        if (object != nullptr)
        {
            objects.push_back(std::move(object));
        }
    }
    void Update()
    {
        for (const auto& object : objects)
        {
            object->Update();
        }
    }
    void Render() const
    {
        for (const auto& object : objects)
        {
            object->Render();
        }
    }
    void CloneObject(std::size_t index)
    {
        if (index < objects.size())
        {
            objects.push_back(objects[index]->Clone());
        }
    }
private:
    std::vector<std::unique_ptr<GameObject>> objects;
};
```





- 상속은 일반화 관계와 대체 가능성 표현
- public 상속은 is-a 관계에 사용
- 기반 클래스의 공개 인터페이스와 의미를 파생 클래스도 유지해야 함
- 파생 클래스는 직접 기반 클래스 생성자를 초기화 리스트에서 선택
- 생성은 기반 클래스에서 파생 클래스로 진행
- 소멸은 파생 클래스에서 기반 클래스로 진행
- 초기화 순서는 초기화 리스트의 작성 순서가 아니라 선언 구조로 결정
- 같은 이름의 함수 선언은 기반 클래스의 오버로드 집합을 숨길 수 있음
- using 선언으로 기반 함수들을 다시 노출 가능
- 재정의의 함수에는 override를 일관되게 사용



- 가상 함수는 실제 객체의 동적 타입에 맞는 함수 호출
- 생성자, 소멸자 안의 가상 호출은 현재 생성 또는 소멸 중인 클래스
- 가상 함수의 기본 인수는 정적 타입에서 선택
- 추상 클래스는 공통 상태와 일부 구현을 가지면서 계약을 정의
- 작은 인터페이스는 독립적인 역할을 표현
- 다형 기반 클래스는 가상 소멸자 제공
- 객체 찢림을 피하려면 다형 객체는 참조나 포인터로 전달
- 다형 객체 컨테이너는 스마트 포인터 사용
- 반복되는 다운캐스팅은 인터페이스 설계를 재검토해야 한다는 신호
- 다중 상속은 이름 충돌과 다이아몬드 구조에 주의
- 상속은 is-a, 구성은 has-a 관계에 사용